

Content Cockpit: Complete Tech Stack & Implementation Plan

Architecture Overview

Serverless Microservices Architecture

- Multiple Cloud Run services handling specific functions
- Event-driven processing pipeline
- Auto-scaling based on workload
- Pay-per-use cost model

Core Tech Stack

Development Environment

- **IDE:** Cursor Pro (AI-first development)
- **Version Control:** GitHub (code storage & CI/CD)
- **Containerization:** Docker (for Cloud Run deployment)
- **Local Testing:** Docker Compose (multi-service testing)

Cloud Infrastructure (Google Cloud Platform)

- **Compute:** Cloud Run (serverless containers)
- **Storage:** Cloud Storage (media files)
- **Database:**
 - Firestore (metadata, user data)
 - Cloud SQL (structured data if needed)
- **AI/ML:**
 - Speech-to-Text API (transcription)
 - Vertex AI Vector Search (semantic search)
 - Document AI (content analysis)
- **Messaging:** Cloud Tasks (async job processing)
- **Monitoring:** Cloud Logging & Monitoring

Programming Languages & Frameworks

- **Backend:** Python with FastAPI (AI-friendly, great for media processing)

- **Media Processing:** FFmpeg (video/audio manipulation)
- **AI/ML Libraries:**
 - OpenAI/Anthropic SDKs (content analysis)
 - scikit-learn (classification)
 - sentence-transformers (embeddings)

Service Architecture

1. Media Ingestion Service

Purpose: Handle file uploads and initial processing

- File validation and virus scanning
- Metadata extraction
- Storage in Cloud Storage
- Trigger downstream processing

Endpoints:

- `POST /upload` - Upload media files
- `GET /status/{job_id}` - Check processing status

2. Audio Processing Service

Purpose: Extract and transcribe audio

- Audio extraction from video files
- Audio enhancement/cleanup
- Speech-to-text conversion
- Speaker identification (if multiple speakers)

Triggers: Cloud Task from Media Ingestion **Outputs:** Transcript with timestamps

3. Video Processing Service

Purpose: Video manipulation and analysis

- Green screen background removal
- Scene detection and segmentation
- Frame extraction for analysis

- Video quality optimization

Triggers: Cloud Task from Media Ingestion **Outputs:** Processed video files, scene metadata

4. Content Analysis Service

Purpose: AI-powered content understanding

- Transcript analysis for yoga concepts
- Philosophical theme extraction
- Asana identification and timestamps
- Vector embedding generation

Triggers: Cloud Task after transcription complete **Outputs:** Structured content metadata, embeddings

5. Search & Query Service

Purpose: Intelligent content discovery

- Vector similarity search
- Concept-based filtering
- Timestamp-based queries
- Content recommendation

Endpoints:

- `POST /search` - Search content by concepts
- `GET /asanas` - List all identified asanas
- `POST /clips` - Generate content clips

6. Content Generation Service

Purpose: Create derivative content

- Clip extraction based on timestamps
- Multi-format export (social media, podcasts)
- Automated content packaging
- Thumbnail generation

Implementation Phases

Phase 1: Foundation (Week 1-2)

Set up development environment:

1. Configure external SSD development environment
2. Set up Google Cloud Project and billing
3. Create GitHub repository
4. Install and configure Cursor Pro with Google Cloud SDK

First Service - Media Ingestion:

1. Create basic FastAPI service
2. Implement file upload to Cloud Storage
3. Add basic metadata extraction
4. Deploy to Cloud Run
5. Test with sample yoga video

Phase 2: Core Processing (Week 3-4)

Audio Processing Service:

1. Build audio extraction using FFmpeg
2. Integrate Google Speech-to-Text API
3. Add speaker identification
4. Store transcripts in Firestore

Video Processing Service:

1. Implement green screen removal
2. Add scene detection
3. Extract key frames
4. Store processed videos

Phase 3: Intelligence Layer (Week 5-6)

Content Analysis Service:

1. Build concept extraction using LLM APIs
2. Implement asana identification
3. Create vector embeddings

4. Set up Vertex AI Vector Search

Search Service:

1. Build semantic search capabilities
2. Add filtering and querying
3. Implement recommendation engine

Phase 4: Content Generation (Week 7-8)

Content Generation Service:

1. Build clip extraction tool
2. Add multi-format export
3. Implement automated packaging
4. Create thumbnail generation

Frontend Dashboard:

1. Simple web interface for management
2. Upload and monitoring capabilities
3. Search and preview functionality
4. Export and download features

First Steps (This Week)

Step 1: Environment Setup (Day 1)

In Cursor, ask it to help you:

1. Set up Google Cloud Project
2. Enable required APIs (Cloud Run, Storage, Speech-to-Text, etc.)
3. Create service account with proper permissions
4. Install Google Cloud SDK on external drive

Step 2: Create First Service (Day 2-3)

Prompt Cursor with:

"Create a Python FastAPI service that can receive video file uploads, store them in Google Cloud Storage, and return an upload confirmation with a unique job ID. Include proper error handling and logging."

Step 3: Containerization (Day 4)

Ask Cursor to:

1. Create Dockerfile for the service
2. Set up docker-compose for local testing
3. Configure Cloud Run deployment

Step 4: Test Deploy (Day 5)

Deploy your first service:

1. Build and test locally
2. Deploy to Cloud Run
3. Test with a sample yoga video
4. Verify end-to-end upload flow

Sample Cursor Prompts to Get Started

Environment Setup

"I need to set up a Google Cloud project for my Content Cockpit application. Help me:

1. Create a new GCP project
2. Enable Cloud Run, Cloud Storage, Speech-to-Text, and Firestore APIs
3. Create a service account with the minimal required permissions
4. Set up authentication for local development

Provide step-by-step instructions I can follow."

First Service Creation

"Create a Python FastAPI microservice for media file uploads with these requirements:

- Accept video/audio file uploads (mp4, mov, wav, mp3)
- Validate file types and sizes (max 500MB)
- Upload files to Google Cloud Storage with organized folder structure
- Return a unique job ID for tracking
- Include proper error handling and logging
- Make it ready for Cloud Run deployment"

Deployment Setup

"Help me containerize this FastAPI service and deploy it to Cloud Run:

1. Create an optimized Dockerfile

2. Set up docker-compose for local testing
3. Create Cloud Run deployment configuration
4. Set up environment variables for production
5. Configure automatic scaling parameters"

Cost Estimation (Light Usage)

Google Cloud (per month):

- Cloud Run: ~\$5-20 (depending on processing volume)
- Cloud Storage: ~\$2-5 (for media files)
- Speech-to-Text: ~\$10-30 (based on hours transcribed)
- Firestore: ~\$1-5 (metadata storage)
- **Total: ~\$20-60/month** for moderate usage

Development Tools:

- Cursor Pro: \$20/month
- **Total Monthly: ~\$40-80**

Success Metrics

Phase 1: Successfully upload and store a yoga video **Phase 2:** Generate accurate transcript with timestamps

Phase 3: Search and find specific asanas or concepts **Phase 4:** Export a clean 30-second clip about a specific topic

This plan gives you a clear roadmap from zero to a fully functional Content Cockpit, with each phase building on the previous one and delivering concrete value.